

## **BC Experiment-8**

**Aim:** To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

### **Theory:**

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

### Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

### Steps to Connect Ganache with Metamask and Remix IDE:

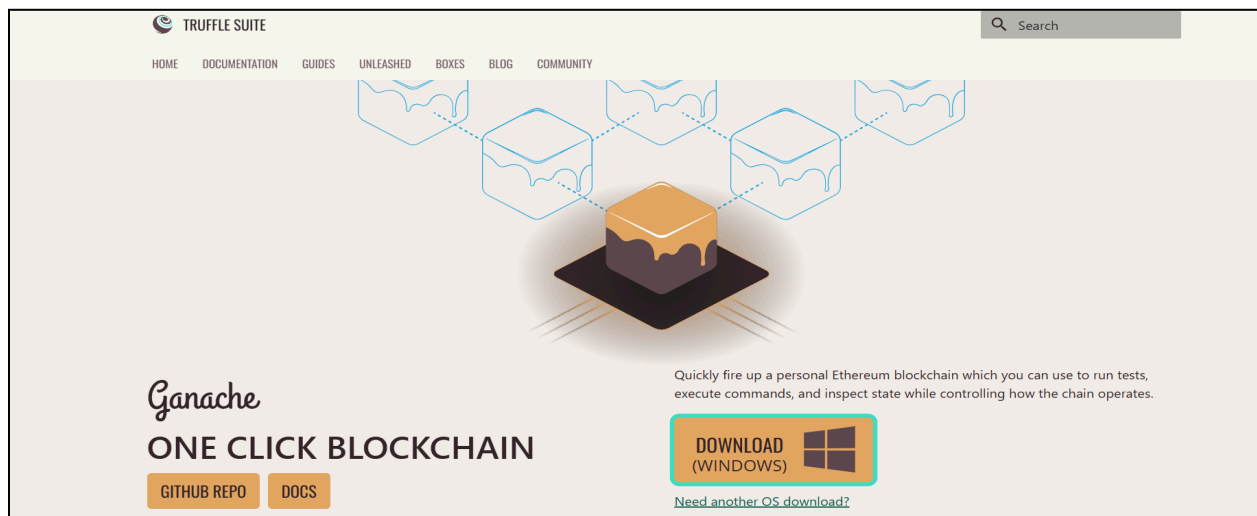
1. Install Ganache:
  - Download and install Ganache from the Truffle Suite website.
  - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
  - Open Metamask and switch to a Custom RPC Network.
  - Enter the Ganache RPC server URL (e.g., [HTTP://127.0.0.1:7545](http://127.0.0.1:7545)).
  - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
  - Open Remix IDE in the browser.
  - Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
  - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
  - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
contract SimpleStorage {
    uint public data;
    function set(uint x) public {
        data = x;
    }
}
```

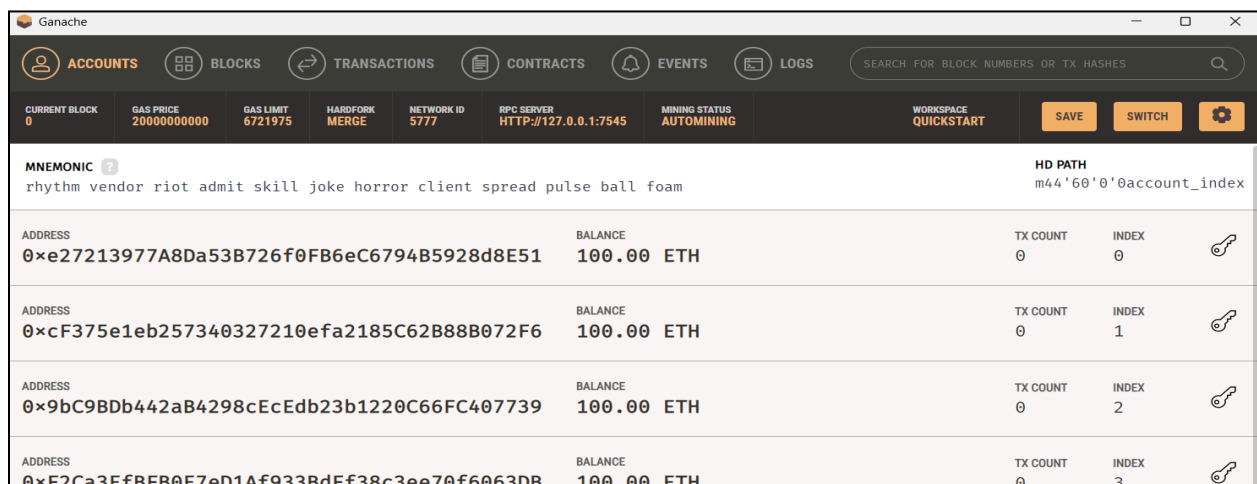
5. Compile and Deploy the Contract:
  - Compile using Remix.
  - Deploy via the Metamask account connected to Ganache.
  - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
  - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
  - Use Remix to call contract functions (**set()** or **get()**).
  - Observe the state changes and corresponding transactions on Ganache.

## Implementation:

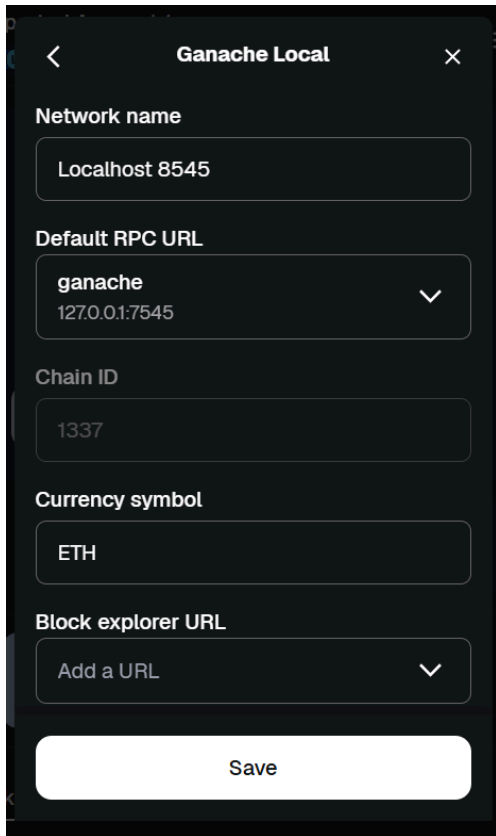
### Step 1: Download and install ganache



Once installed, open ganache and click on Quickstart Ethereum to start localhost.



Step 2: In MetaMask, click on the networks tab and add a custom network.



Network name

Localhost 8545

Default RPC URL

ganache  
127.0.0.1:7545

Chain ID

1337

Currency symbol

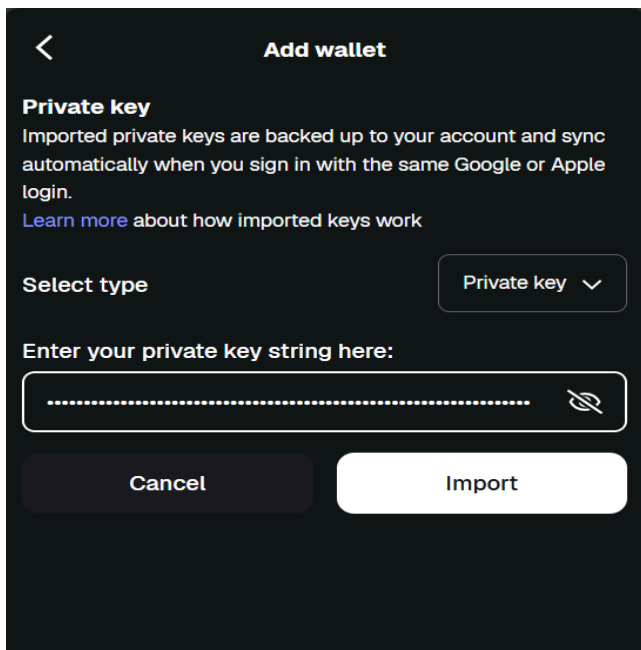
ETH

Block explorer URL

Add a URL

Save

Step 3: Import ganache account into metamask



Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.

[Learn more](#) about how imported keys work

Select type

Private key

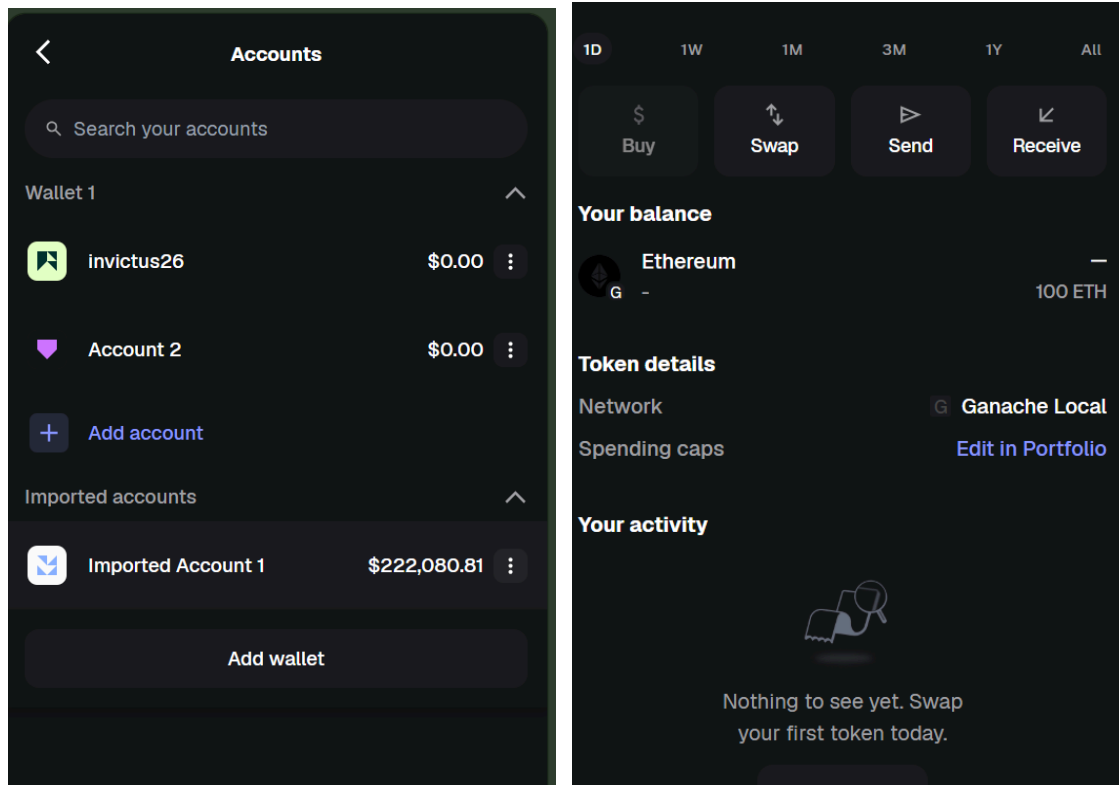
Enter your private key string here:

.....

Cancel

Import

## Wallet balance appears in metaMask account



Step 4: In Remix IDE, create a new file **crowdfunding.sol**

Code:

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.0;
```

```
contract Crowdfunding {  
    address public owner;  
    uint public goal;  
    uint public deadline;  
    uint public totalFunds;
```

```
    mapping(address => uint) public contributions;
```

```
    constructor(uint _goal, uint _duration) {  
        owner = msg.sender;  
        goal = _goal;  
        deadline = block.timestamp + _duration;
```

```

}

function contribute() public payable {
    require(block.timestamp < deadline, "Deadline passed");
    require(msg.value > 0, "Send some ETH");

    contributions[msg.sender] += msg.value;
    totalFunds += msg.value;
}

function withdrawFunds() public {
    require(msg.sender == owner, "Only owner");
    require(totalFunds >= goal, "Goal not reached");

    (bool success, ) = payable(owner).call{value: totalFunds}("");
    require(success, "Transfer failed");
}

function refund() public {
    require(block.timestamp > deadline, "Not ended");
    require(totalFunds < goal, "Goal was reached");

    uint amount = contributions[msg.sender];
    require(amount > 0, "No contribution");

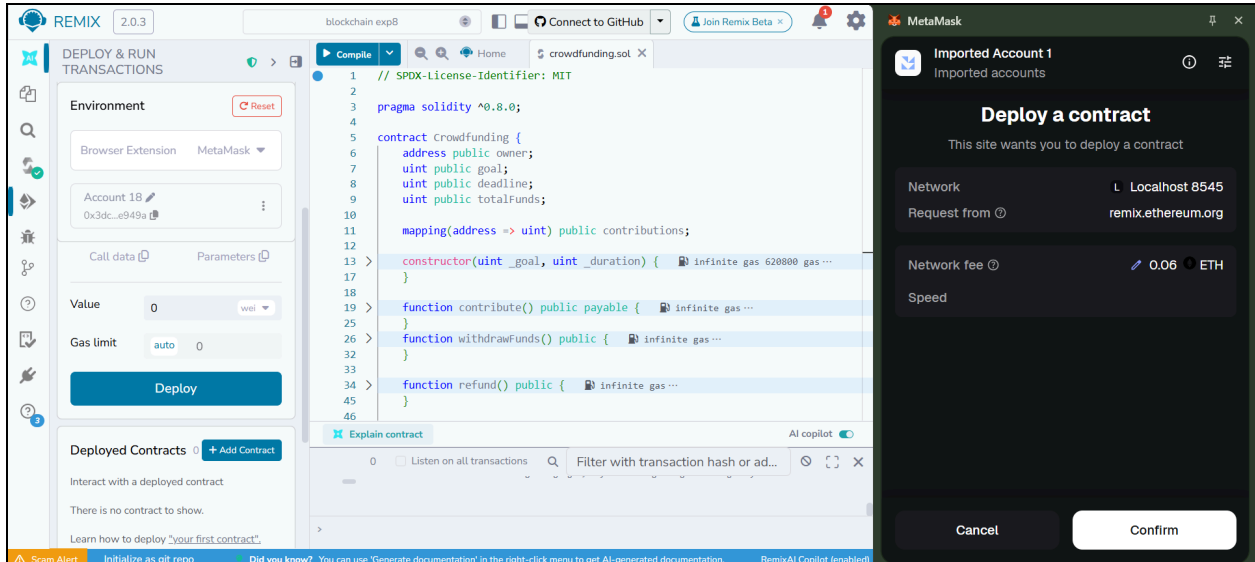
    contributions[msg.sender] = 0;

    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "Refund failed");
}

function getBalance() public view returns (uint) {
    return address(this).balance;
}
}

```

Step 5: Compile and run the contract.  
 When prompted in metamask, confirm the transaction.



The contract is deployed!

The deducted amount can be seen in ganache app along with transaction count

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK  
2

GAS PRICE  
20000000000

GAS LIMIT  
6721975

HARDFORK  
MERGE

NETWORK ID  
5777

RPC SERVER  
HTTP://127.0.0.1:7545

MINING STATUS  
AUTOMINING

WORKSPACE  
QUICKSTART

SAVE

SWITCH

MNEMONIC 7

apology essence feed north arm announce receive axis total wall chase involve

HD PATH

m44'60'0"8account\_index

ADDRESS

0x678aB28b947030404778C6193423B73c7a014b18

BALANCE

99.92 ETH

TX COUNT

2

INDEX

0

ADDRESS

0xcC163421b1091647e96D5Ce9ddEdF1e624dad0a6

BALANCE

100.00 ETH

TX COUNT

0

INDEX

1

ADDRESS

0x7Ea6E7307F61C1b29eA2FC7c40F16e602b514a31

BALANCE

100.00 ETH

TX COUNT

0

INDEX

2

ADDRESS

0xb00C1B0A9e7E90b2b538A98d7DD98A499578F4dd

BALANCE

100.00 ETH

TX COUNT

0

INDEX

3

ADDRESS

0x176239f7D57783b2Fe396cb7FA3B73178C750713

BALANCE

100.00 ETH

TX COUNT

0

INDEX

4

ADDRESS

0x9BFCFd4b9223c4d5d551bb8152DBAFFAF449A36B

BALANCE

100.00 ETH

TX COUNT

0

INDEX

5

**Conclusion:**

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.